

DocuSense AI

Prompt Engineering Guide

All 8 Project Prompts · *ROLE · CONTEXT · TASK · FORMAT · CONSTRAINT* Structure

Team 2 | Hack & Fest AI Innovation Hackathon | SRM IST 2026

Prompt 1 — Project Kickoff & Initial Architecture

ROLE	You are an expert Python full-stack AI developer specializing in local LLM pipelines and Streamlit applications.
CONTEXT	SRM IST Hack & Fest AI Innovation Hackathon (Day 4). Problem Statement 1.2: Contract & Policy Clause Explainer. Ollama local models: llama3.2:latest and nomic-embed-text:latest. No internet APIs — 100% offline and local.
TASK	Convert an existing Stitch MCP UI design (ClauseGuard AI) into a fully functional Streamlit application with PDF upload, document processing, clause analysis, RAG-powered Q&A, and risk detection.
FORMAT	Python code using Streamlit for UI, PyPDF2 for PDF reading, and Ollama REST API for LLM calls. Modular files: app.py, llm_utils.py, pdf_utils.py, rag_utils.py.
CONSTRAINT	Must work 100% offline. No OpenAI, no Hugging Face API, no paid services. Only Ollama local models.

Prompt 2 — PDF Audit Report Export

ROLE	You are a Python developer with expertise in ReportLab PDF generation and Streamlit download buttons.
CONTEXT	DocuSense AI currently only exports audit reports as .md (Markdown) files. Users need a downloadable PDF format for professional presentation.
TASK	Add a PDF export button to the Streamlit application that generates a professional audit report using ReportLab and allows the user to download it.
FORMAT	Create a new pdf_generator.py module. Add a st.download_button in app.py. The PDF must include document summary, clauses, risk scores, and safety verdict.
CONSTRAINT	Do not change any existing business logic. Only add the PDF generation layer. The PDF must be readable on Windows without installing additional font software.

Prompt 3 — Enterprise UI Redesign & Safety Verdict System

ROLE	You are a senior UI/UX designer and Streamlit developer who builds professional, enterprise-grade data dashboards.
CONTEXT	The current Stitch-based theme uses generic colors that look unprofessional. The summary tab needs a clear Accept/Reject recommendation for any policy or document.
TASK	Redesign the entire UI with a professional dark theme. Add a Policy Safety Score (0-100%) and a Safety Verdict banner (SAFE TO SIGN, PROCEED WITH CAUTION, UNSAFE TO SIGN) based on risk score analysis. Fix the Q&A; tab so it actually returns answers from the document.
FORMAT	Custom CSS injected via st.markdown(). Safety verdict displayed as a styled banner. Score shown as st.metric and st.progress.
CONSTRAINT	Colors must look professional — no primary red/blue/green. Use dark backgrounds (#0F1923, #1A2332) with accent gradients. Q&A; must use the uploaded document as its only source — no hallucination.

Prompt 4 — Multi-Input Support, Indic Language Translation & Rebranding

ROLE	You are a multilingual AI engineer experienced with Indic language NLP, Streamlit UI components, and RAG pipelines.
CONTEXT	DocuSense AI currently only accepts PDF files. Many users are non-English speakers from South India who cannot understand English legal documents. The app is named ClauseGuard AI which is hard to remember.
TASK	1. Rename the app to DocuSense AI. 2. Add three input modes: PDF upload, Image upload (OCR), and raw text paste. 3. Add language selection (English, Telugu, Tamil, Malayalam, Hindi, Spanish, French). 4. Translate all outputs to the selected language. 5. Add a text input box in the Q&A; tab for custom questions.
FORMAT	Language selector as st.radio or styled pill buttons. Translation call added to all LLM functions in llm_utils.py. Raw text input as st.text_area in the document upload section.
CONSTRAINT	Numbers, currency (Rs.45,00,000), proper names (Palanisamy, Muthusamy), survey numbers (123/2A), and document IDs must always remain in original English digits — never translated or transliterated.

Prompt 5 — EasyOCR Engine & Document Legality Forensics

ROLE	You are a computer vision and OCR specialist with expertise in Python image processing and Windows system compatibility.
CONTEXT	The app was using Tesseract-OCR which is not installed. When switching to EasyOCR, a Windows console encoding crash occurs: 'charmap' codec can't encode character in position 12. Users also need the app to detect if an uploaded document is legally valid or fraudulent.
TASK	1. Replace Tesseract with EasyOCR. 2. Fix the Windows CP1252 encoding crash by wrapping stdout/stderr with UTF-8 TextIOWrapper and setting verbose=False. 3. Add a check_document_legality() function that analyzes the document for mandatory legal fields and returns a legality verdict.
FORMAT	Fix in pdf_utils.py using io.TextIOWrapper. New function check_document_legality() in llm_utils.py returning a dict with: verdict, confidence_score, present_fields, missing_fields, and risk_flags. Display in a new Legality Check card in app.py.
CONSTRAINT	EasyOCR must not produce any Unicode errors on Windows. The legality verdict must be derived strictly from the uploaded document content only — not from general knowledge.

Prompt 6 — RAG Grounding Fix, Indic PDF Font Support & Sample Deed

ROLE	You are an expert in LLM prompt engineering, RAG grounding, ReportLab PDF typography, and Indian legal document formats.
CONTEXT	When a user asks 'Who is the CEO of Google?' after uploading a land deed, the model hallucinated a Tamil answer. Also, the PDF report generates black rectangle glyphs instead of Indic script because Helvetica doesn't support Unicode. The sample document needs a realistic South Indian land sale agreement.
TASK	1. Enforce strict RAG grounding: if a question cannot be answered from the uploaded document, the model must respond 'This information is not mentioned in the uploaded document.' 2. Fix PDF Indic font rendering by registering Windows Segoe UI TTF font. 3. Sanitize emoji characters from PDF titles. 4. Update sample_data.py with a Kinathukadavu Land Sale Agreement (Palanisamy to Muthusamy, Rs.45,00,000, 2.50 Acres, Survey 123/2A).
FORMAT	Grounding rules added as system instructions in every LLM prompt in llm_utils.py. Font registered in pdf_generator.py header. Sample text as a multi-line string constant in sample_data.py.
CONSTRAINT	The model must NEVER fabricate information not in the uploaded document. Currency amounts, names, and survey numbers must stay in their original English/numeric form even when output language is Tamil, Telugu, or Malayalam.

Prompt 7 — Q&A; UI Overhaul, Full-Context RAG & README Overhaul

ROLE	You are a senior full-stack developer and technical documentation writer specializing in Streamlit UX and GitHub README standards.
CONTEXT	Users cannot find the Q&A; text box in the app. The RAG answers are incomplete because chunk-based retrieval splits seller/buyer names across chunk boundaries. The README file needs to be overhauled with team details, system architecture, and feature documentation for hackathon submission.
TASK	1. Make the Q&A; text input box clearly visible and prominent in Tab 4. 2. Switch from chunk-based retrieval to full-context passing (all clauses + full text) for short documents (1-10 pages). 3. Completely overhaul README.md with: Team 2 member roster table, Mermaid architecture diagram, feature matrix table, and step-by-step setup guide.
FORMAT	Q&A; as a visible st.form with st.text_input and st.form_submit_button. README in GitHub-flavored Markdown with Mermaid code block for the architecture diagram.
CONSTRAINT	Do not change any analysis logic or LLM model settings. Only UI layout and context passing method. Full-context passing must preserve all monetary values and proper names in original form.

Prompt 8 — Prompt History Export for Evaluation

ROLE	You are a technical documentation specialist extracting conversation history for academic evaluation and hackathon submission.
CONTEXT	All development prompts used during DocuSense AI's hackathon build session need to be documented for evaluation and PDF conversion. The user needs a clean, structured text file of all 8 development stages.
TASK	Extract all user prompts from the conversation in chronological order, group them by development stage, and write them to a clean .txt file (project_prompts_history.txt) in the project directory.
FORMAT	Plain text file with stage headers separated by dashed lines. Each stage labeled with 'Stage N: [Title]' and followed by the original prompt text. Unicode-safe (UTF-8 encoding).
CONSTRAINT	Output must be a .txt file only — no .md, no .pdf. Include the original unedited user prompts with typos preserved so it reflects authentic development history.

Summary Table — All Prompts at a Glance

#	Stage	Core Ask	Key Constraint
1	Project Kickoff	Build full Streamlit RAG app	100% offline, Ollama only
2	PDF Export	Add downloadable PDF audit report	No logic changes
3	UI Redesign	Professional dark theme + Safety Verdict	No generic colors
4	Multi-Input & Translation	OCR + Text paste + Indic languages	Preserve numbers/names in English
5	EasyOCR + Legality	Fix OCR crash + Document fraud check	No Windows encoding errors

#	Stage	Core Ask	Key Constraint
6	RAG Grounding + PDF Fonts	Stop hallucination + Fix Indic glyphs	Strict document-only answers
7	Q&A; UI + README	Visible Q&A; box + Full docs overhaul	No logic changes
8	Prompt Export	Extract all prompts to .txt	Plain text only

Generated for DocuSense AI — Team 2, SRM IST Hack & Fest 2026 | Confidential Evaluation Document